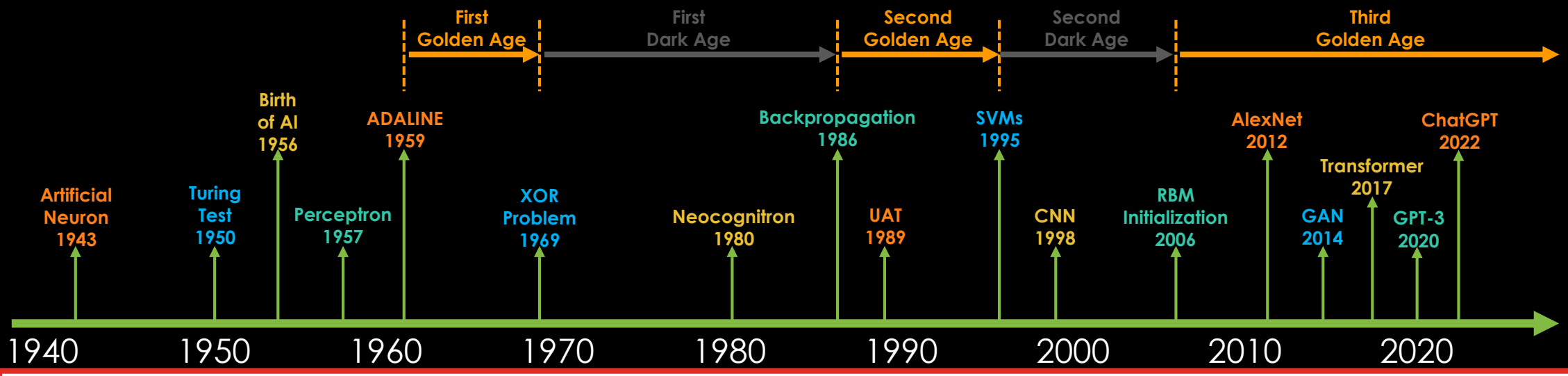
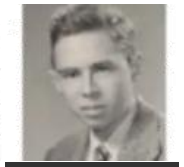
FreeMol-Gen

Molecular Generation from a Language Model Perspective

A Brief History of AI with Deep Learning




McCulloch-Pitts


Rosenblatt


Widrow-Hoff


Minsky-Papert

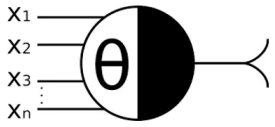

Rumelhart, Hinton et al.

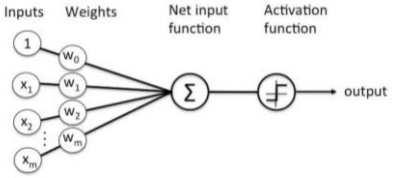

LeCun

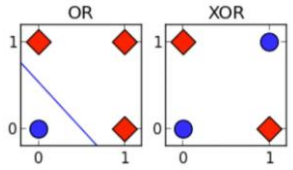

Hinton-Rusaln

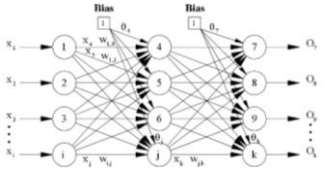

Krizhevsky et al.

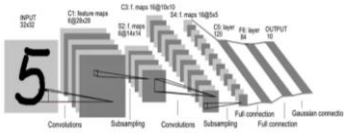

Vaswani

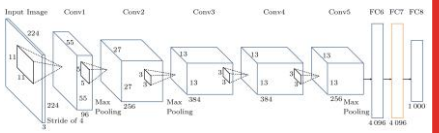






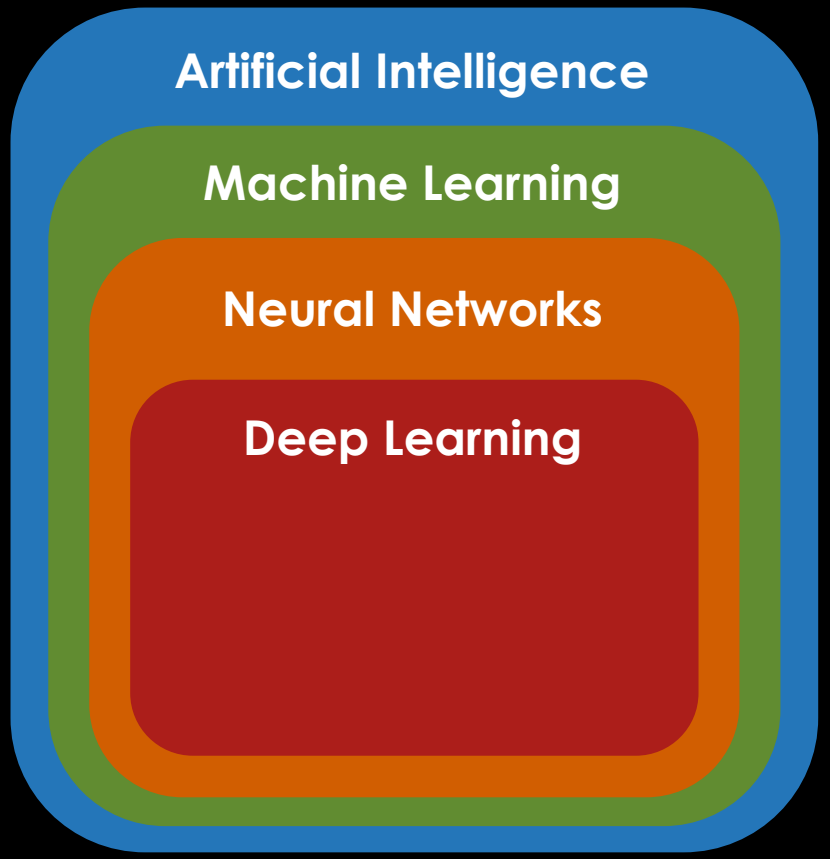




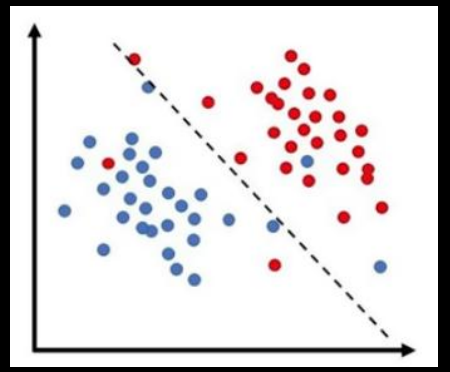


Ref. https://medium.com/@lmpo/a-brief-history-of-ai-with-deep-learning-26f7948bc87b

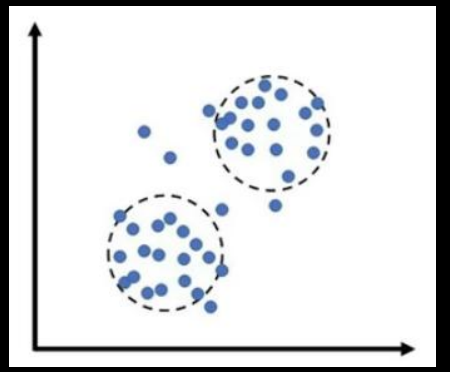
AI Terminology



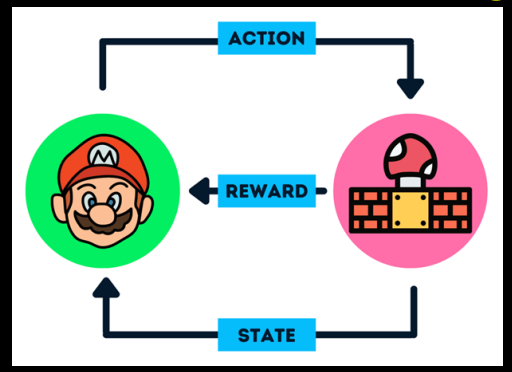
Supervised Learning



Unsupervised Learning



Reinforcement Learning



Ref. 1 https://doi.org/10.3390/s22197655
Ref. 2 https://www.springcanyon.org/?r=50636267091420

How to Generate Text

The "Bible Word-Association" Breakdown

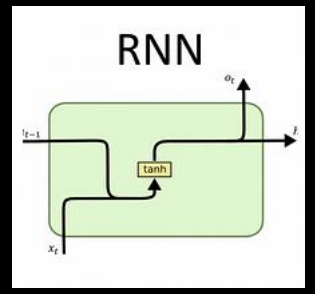
- Phase 1: Pure Randomness (The Blind Guess)
  - Context: No historical data, no context, no rules.
  - Scenario: Putting all 13,000 unique words from the Bible into a lottery drum and drawing one blindly.
  - Probability: Equal distribution for every single word ($P = \frac{1}{13000} \approx 0.007\%$)
  - Outcome: Tabernacle cucumber Nebuchadnezzar generation sparrow...
- Phase 2: Prior Probability (Static Statistics)
  - Context: Analyzing the frequency of 790,000 total words.
  - Scenario: Picking words based solely on how common they are in the Bible.
  - Probability: "The" (~8%), "And" (~5%), and "Of" (~4%) dominate the lottery.
  - Outcome: The and of to and the...
- Phase 3: Posterior Probability (Context-Aware)
  - Context: A new piece of evidence is introduced - the current word is "The".
  - Scenario: Dynamically updating the next word's odds based on the context.
  - Probability: $P(Lord \mid The)$ skyrockets to $\sim 10\%$; grammatically incorrect words drop to 0%.
  - Outcome: The Lord is my shepherd...

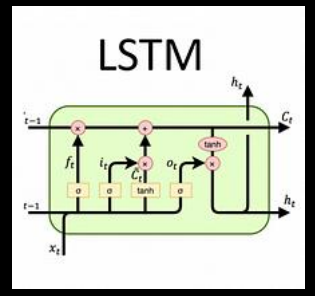
Evolution of Solutions for Sequence Data

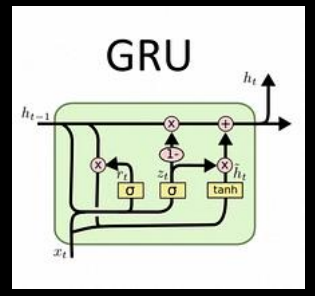
- RNN (Recurrent Neural Network)
  - Hidden state memory
  - Step-by-step processing
  - Hnadles sequential data
- LSTM (Long Short-Term Memory)
  - Complex gates
  - Selective memory
  - Fixes vanishing gradient
- GRU (Gated Recurrent Unit)
  - Simplified gates
  - Faster training & lower compute cost

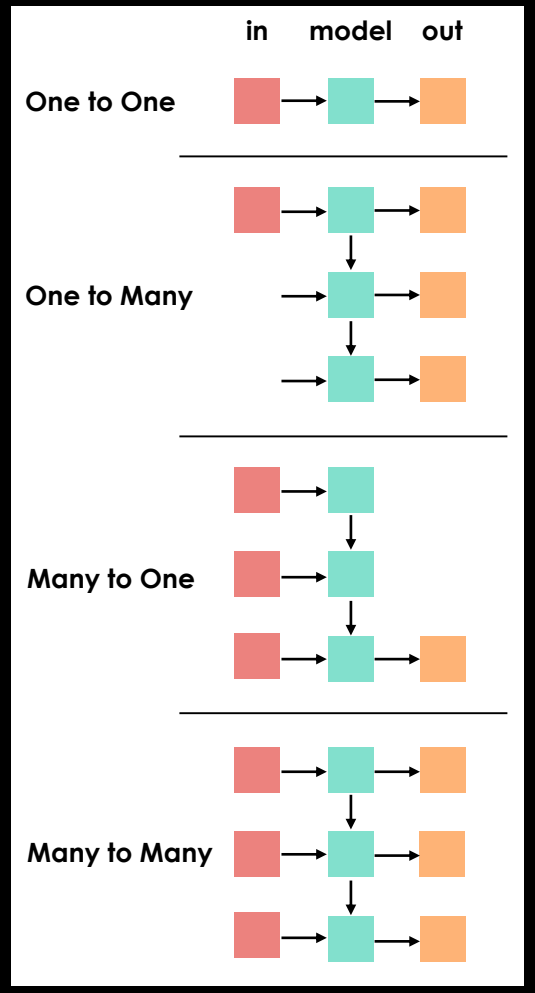
The Common Bottlenecks

1. Zero parallelization
2. Slow step-by-step speed
3. Compression data (memory) loss









Ref. https://doi.org/10.1007/s11042-020-10037-x

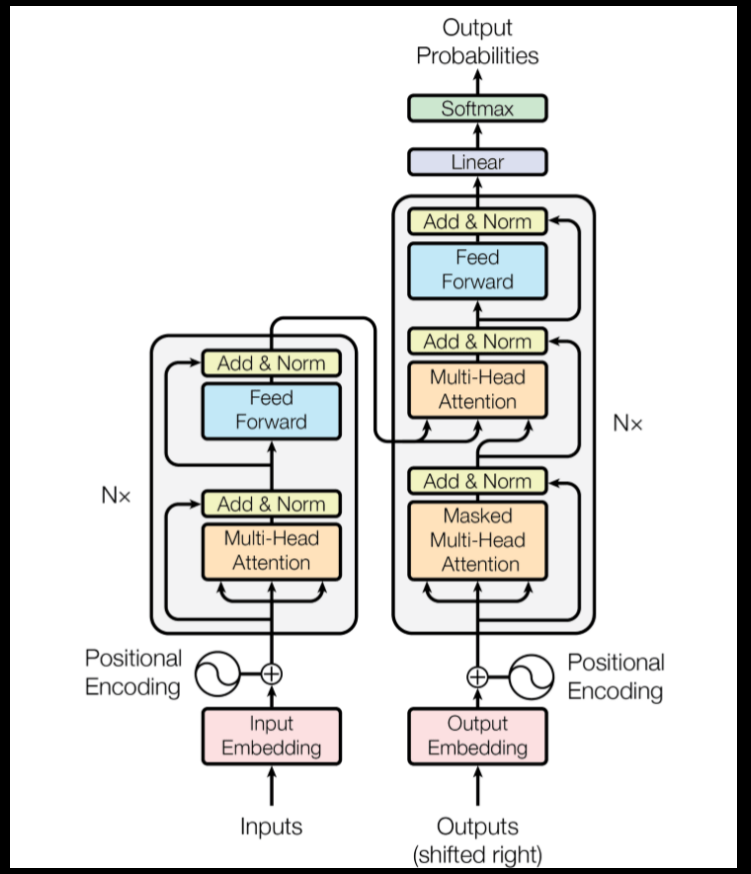
June 12, 2017
The Day AI Changed Forever

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com
Noam Shazeer* Google Brain noam@google.com
Niki Parmar* Google Research nikip@google.com
Jakob Uszkoreit* Google Research usz@google.com
Llion Jones* Google Research llion@google.com
Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu
Łukasz Kaiser* Google Brain lukaszkaiser@google.com
Illia Polosukhin* ‡ illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.



Ref. https://doi.org/10.48550/arXiv.1706.03762

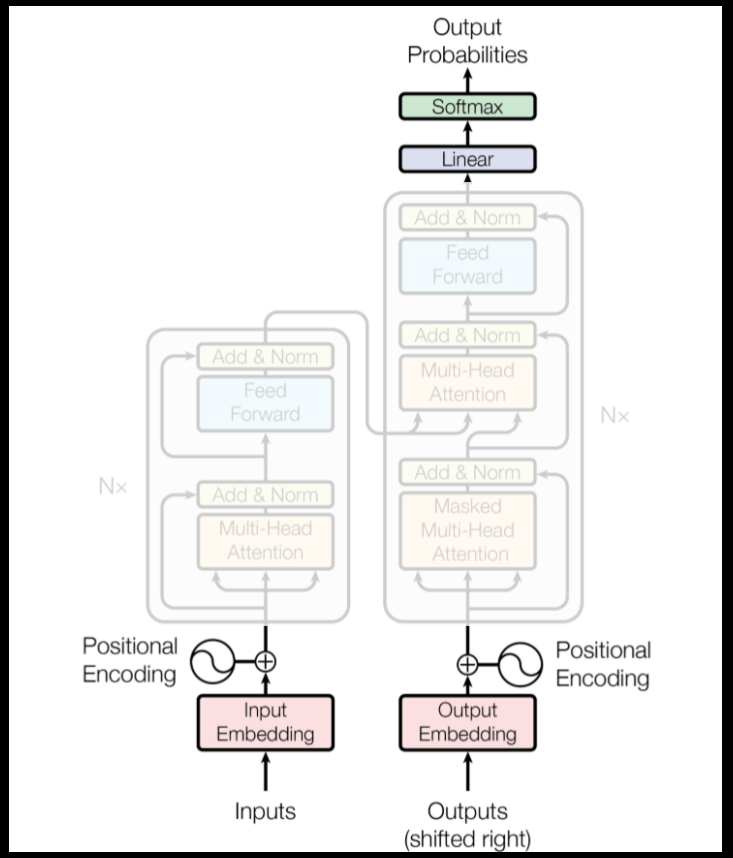
Transformer

Architecture & Component Deep Dive - 1

Input & Output Pipeline

- Input processing (Pre-processing):
  - Tokenizer: Breaks down raw input into discrete numerical ID tokens (e.g., apple => 1, banana => 2, car => 3).
  - Embedding: Maps numerical IDs into high-dimensional vectors to capture semantic or chemical meaning in a continuous vector space.
  - Positional Encoding: Adds explicit "timestamp" vectors to the embeddings so the model knows token order.
- Output processing (Post-processing):
  - Linear & Softmax: Converts high-dimensional decoder output back to a vocabulary-sized probability distribution for the next token prediction.
  - Detokenizer: Translates the predicted numerical ID back into human-readable text or chemical symbols for the final output.

Hints: Apart from Positional Encoding, the rest are designs that are also present in traditional RNNs.

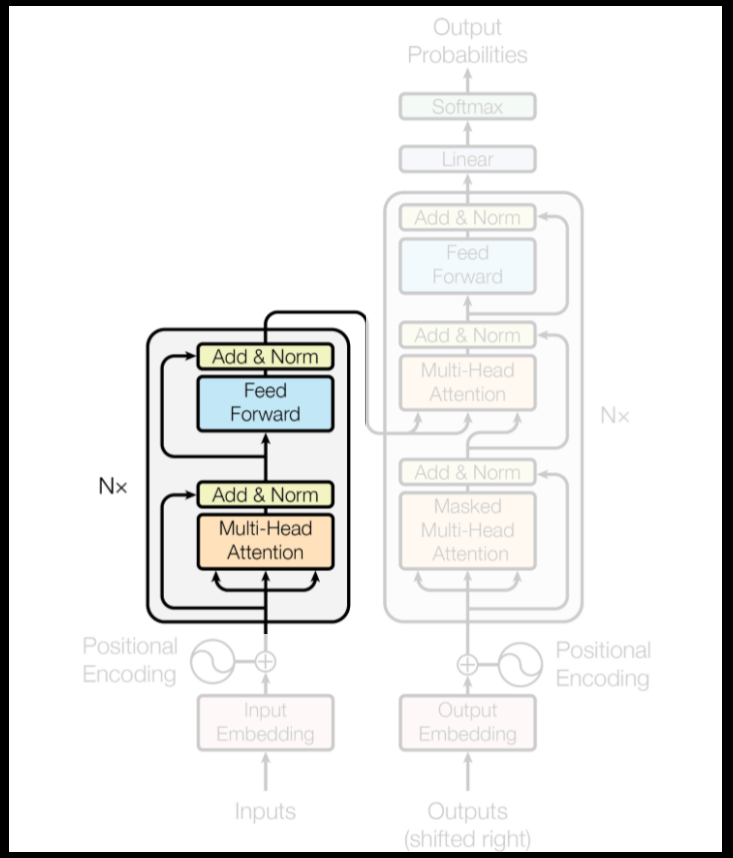


Transformer

Architecture & Component Deep Dive - 2

Core Transformer Block

- Multi-Head Attention (Context Analyzer):
  - What it does: Scans the entire sequence simultaneously to calculate relevance scores between all tokens, deeply analyzing the hidden context.
  - "Multi-Head": Uses multiple "heads" to analyze data from different perspectives simultaneously (e.g., one head for syntax, another for semantic dependencies).
- Add & Norm (Stabilizer):
  - What it does: A critical mechanism applied after MHA and Feed-Forward layers.
  - Residual Connection (Add): Creates a shortcut that adds the original input directly to the output of the layer. This ensures gradients flow through the network, preventing information loss in deep layers.
  - Layer Normalization (Norm): Normalizes the range of activation values. This keeps the internal numbers balanced, speeding up convergence and stabilizing the training process.
- Feed-Forward Network (Reasoner):
  - What it does: A traditional, fully-connected neural network applied independently to each token.
  - Purpose: Applies deep non-linear transformations to the information gathered by attention, acting as a logical reasoning amplifier for abstract patterns.

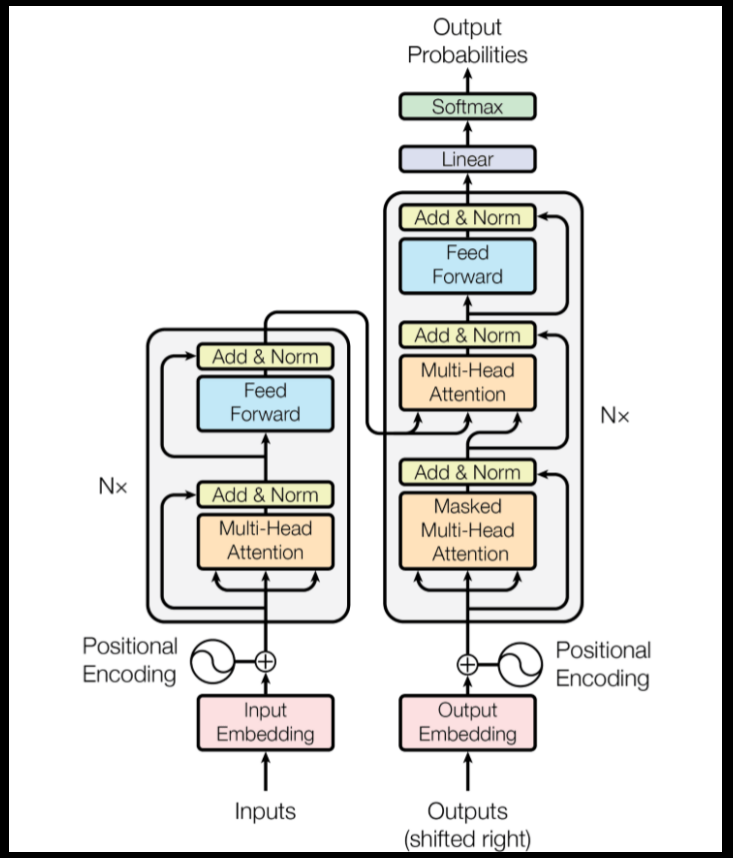


Transformer

Architecture & Component Deep Dive - 3

Encoder-Decoder Architecture (High-Dim Understanding Space)

- Encoder (Understanding and Projection):
  - Workflow: Read the entire input => projects it into a "high-dimensional understanding space" (Latent Memory).
  - Component Logic: Uses Full Self-Attention to connect every word to every other word in the input, deeply contextualizing the entire sequence into a dense latent representation.
- Decoder (Target Generation and Alignment):
  - Workflow: Step-by-step translations from the high-dimensional understanding space back down to the target sequence.
  - Component Logic: To predict the next token, it references two things simultaneously:
    1. Masked Self-Attention: Only looks at its own previously generated results (strictly no-peeking at the future).
    2. Cross-Attention (The Bridge): Actively references the Encoder's high-dimensional projection to ensure the generated output perfectly aligns with the original source context.



Transformer Architectural Trade-Offs

Network Depth (Deeper ≠ Better)	Attention Heads (More ≠ Better)	Tokenizer (Character vs. Word)
• Diminishing returns • Overfitting risk • Slower inference	• Wasted compute • High redundancy • Attention collapse	• Char.: Small dict., needs depth • Word: Shallow model, huge dict.

No universal formula. Architecture design requires iterative trial and error.

Is Everything Just "Word Chain"?

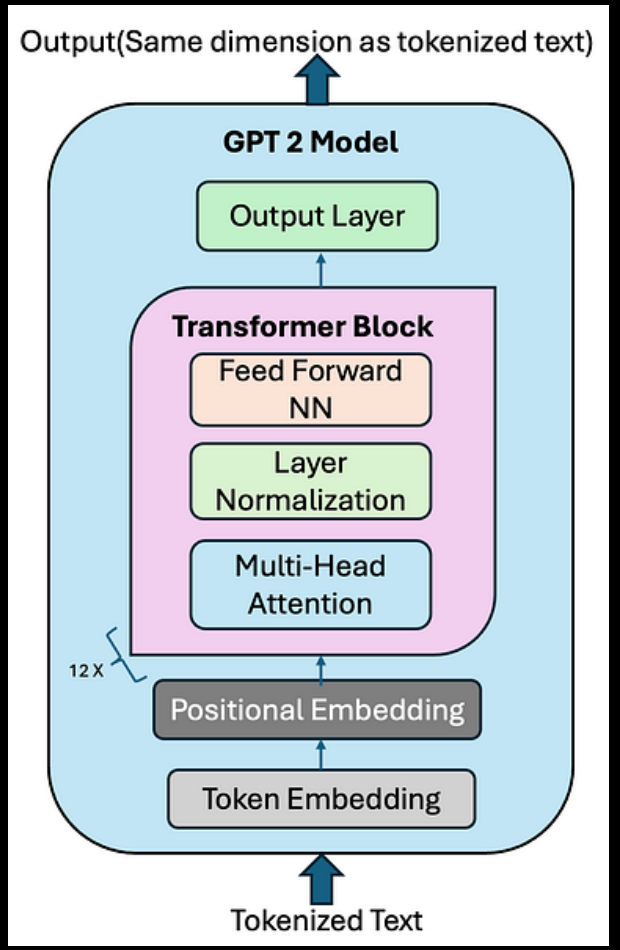
- Rethinking the Encoder:
  Traditional models use a complex "encode-then-decode" structure. Is this dual-layer approach truly necessary?
- The Hidden Strength of Self-Attention:
  The Decoder already possesses built-in Self-Attention, allowing it to independently capture and understand all previous context.
- Simplification:
  Since the Decoder can understand and generate text simultaneously, the redundant Encoder is completely removed.
- Universal Paradigm:
  By treating all tasks as a single continuous sequence, a massive Decoder solves complex problems simply by predicting the next token.

Task Execution by "Word Chain“ (Simple Examples)

- Task: Translation (The Bilingual Pattern)
  - Input: English: Hello => French:
  - GPT Predicts: Bonjour
  - Logic: GPT doesn't actually "understand" grammar. It is simply completing the pattern of a bilingual dictionary it saw during training.
- Task: Sentiment Analysis (Fill-in-the-Blank)
  - Input: Review: "I hated this movie!" => Emotion:
  - GPT Predicts: Negative
  - Logic: It calculates that the word "Negative" has the absolute highest mathematical probability to follow that specific complaint.
- Task: Roleplay / Style Transfer (Probability Shifting)
  - Input: [Role: Pirate] Hello, how are you? =>
  - GPT Predicts: Ahoy, matey!
  - Logic: The [Role: Pirate] tag acts like a filter. It instantly lowers the probability of generating "normal" words and massively boosts the probability of pirate slang.

Why This Works?

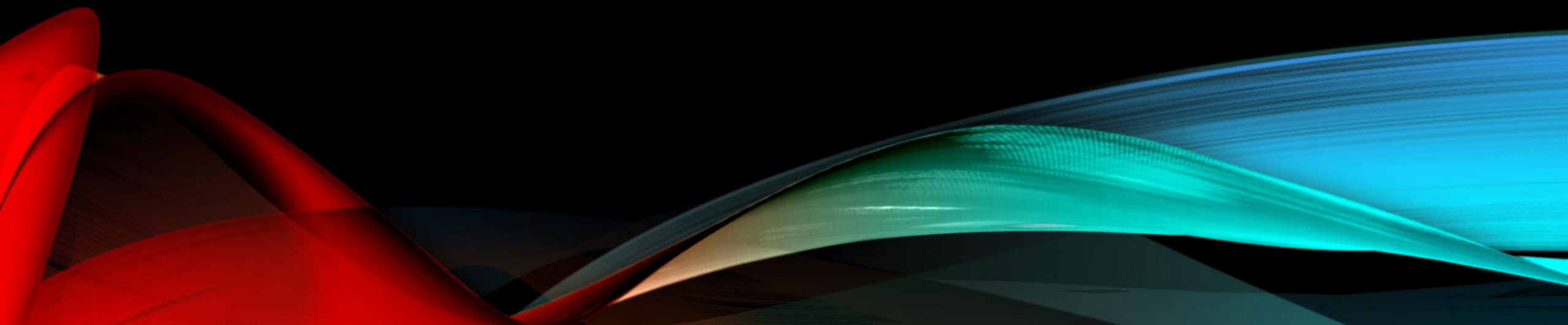
- Massive pre-training teaches the patterns, while Attention Heads dynamically focus on the "Task Tags" to execute the right logic.

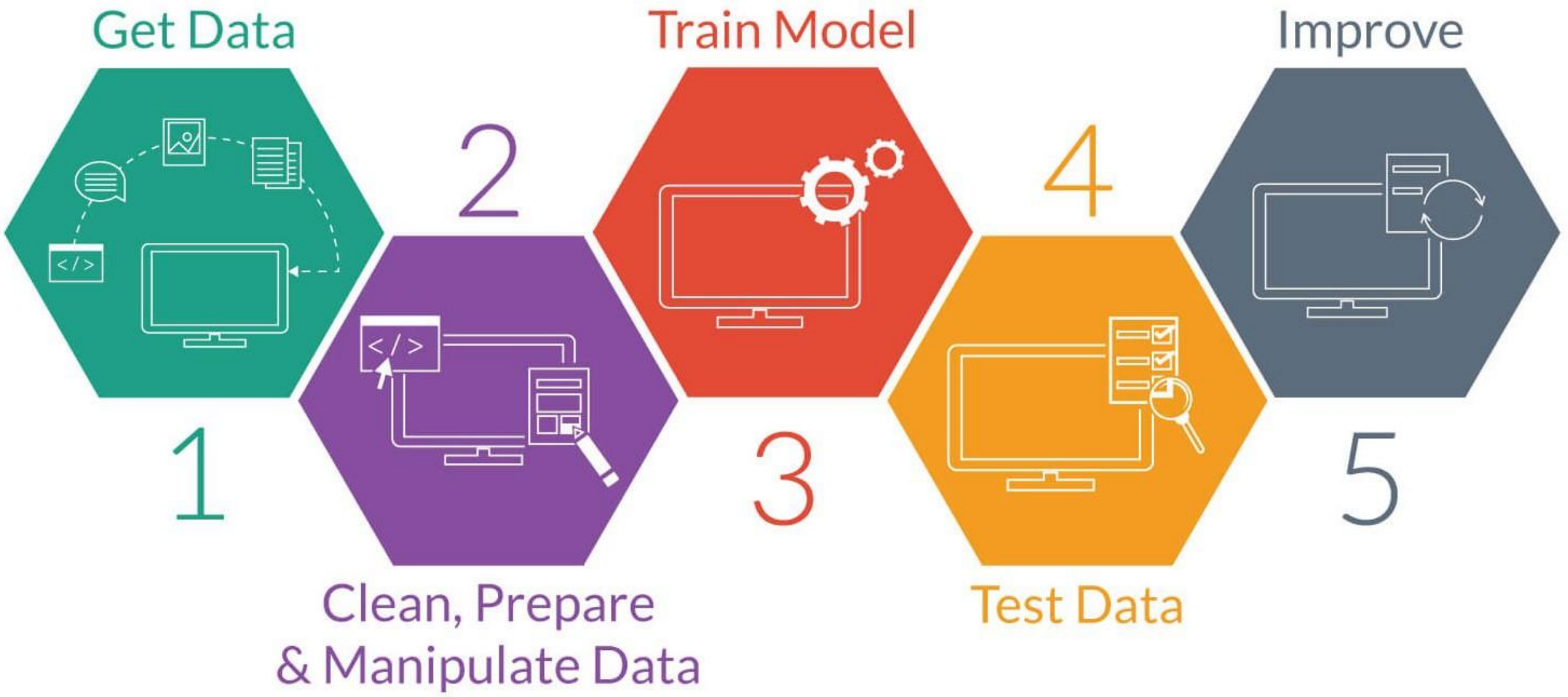


Ref. https://medium.com/@vipul.koti333/from-theory-to-code-step-by-step-implementation-and-code-breakdown-of-gpt-2-model-7bde8d5cecda

Enter FreeMol-Gen

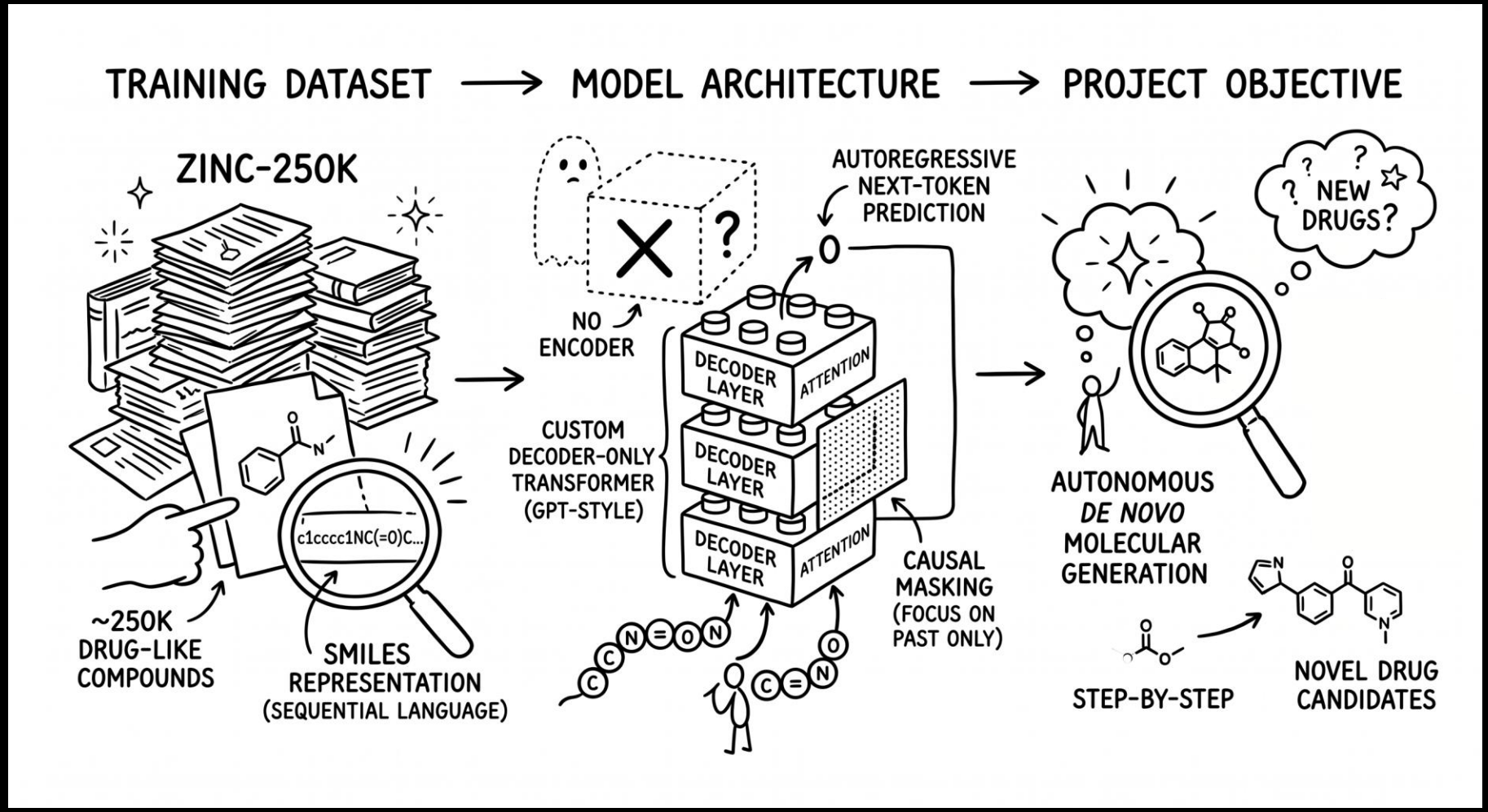
Applying the GPT philosophy to de novo drug design





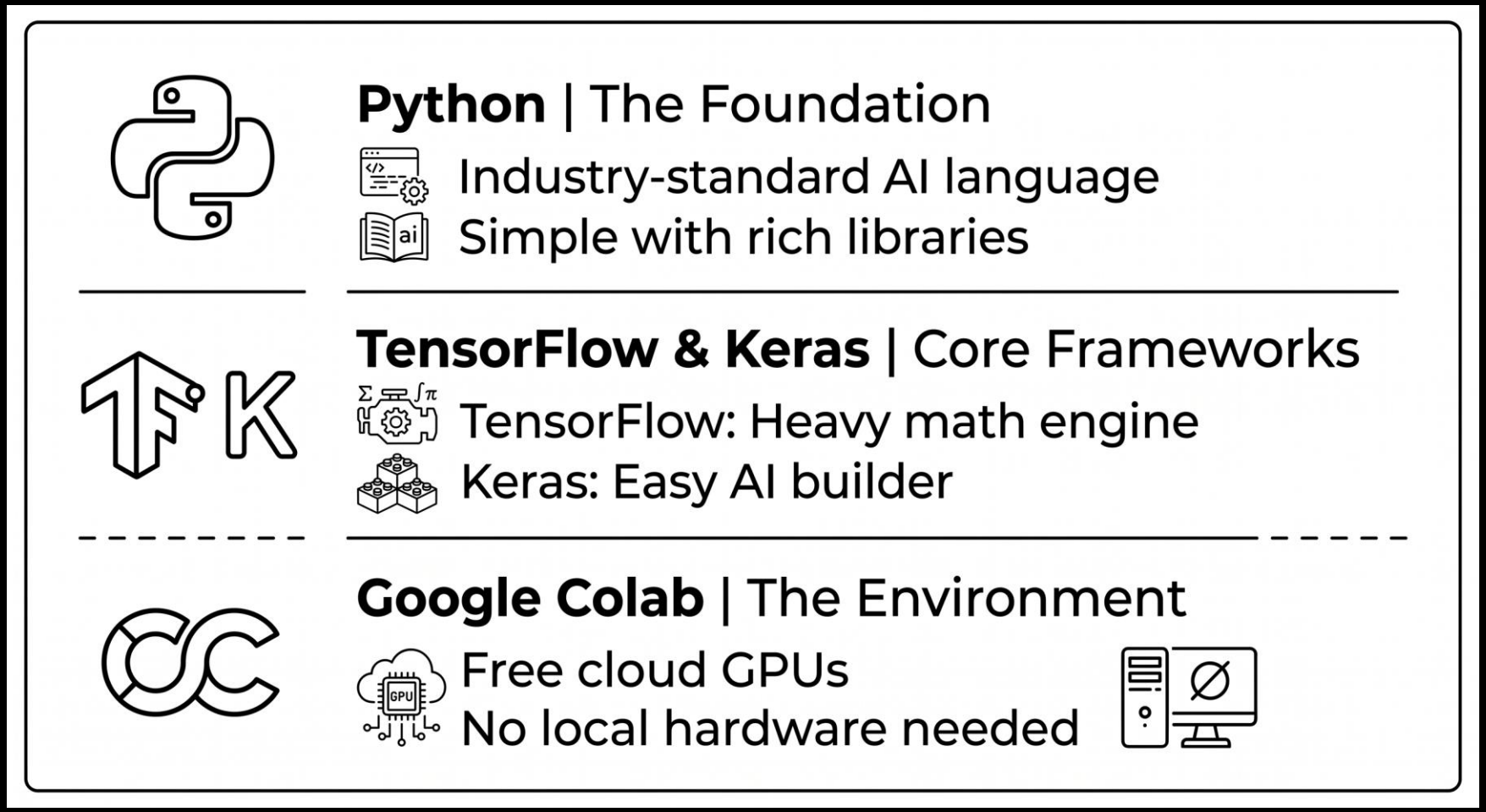
Ref. https://medium.com/@vidishsharma1311/workflow-of-a-machine-learning-project-4eba6875c285

FreeMol-Gen: System Overview



AI Generation [Google. (2026). *Gemini (Nano Banana 2)*.]

Building the Model



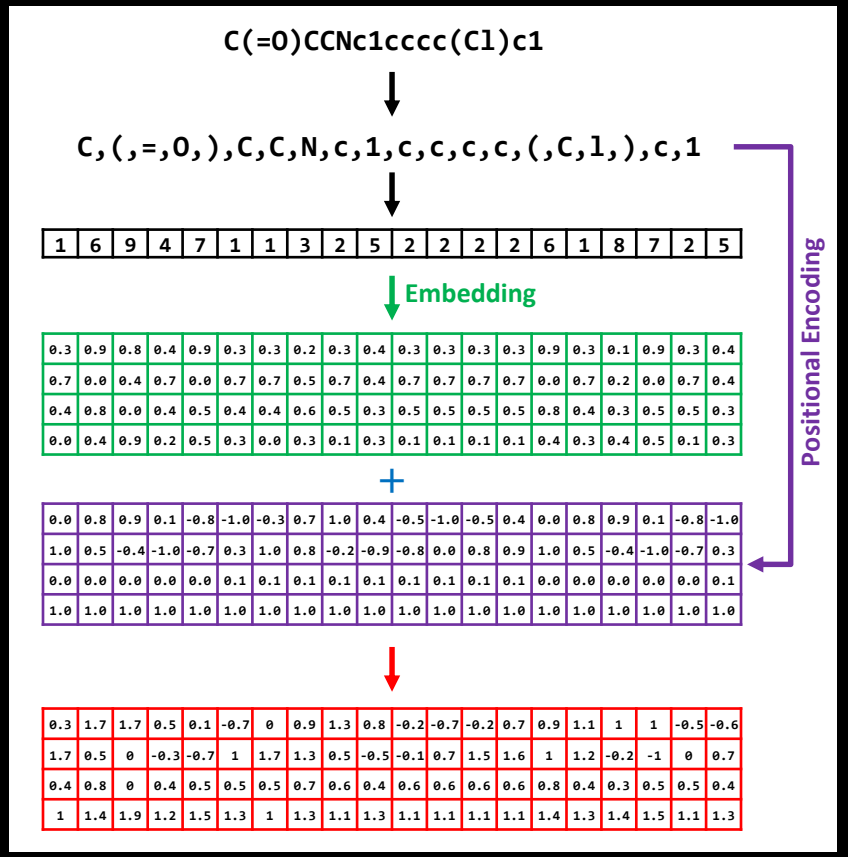
AI Generation [Google. (2026). *Gemini (Nano Banana 2)*.]

Links

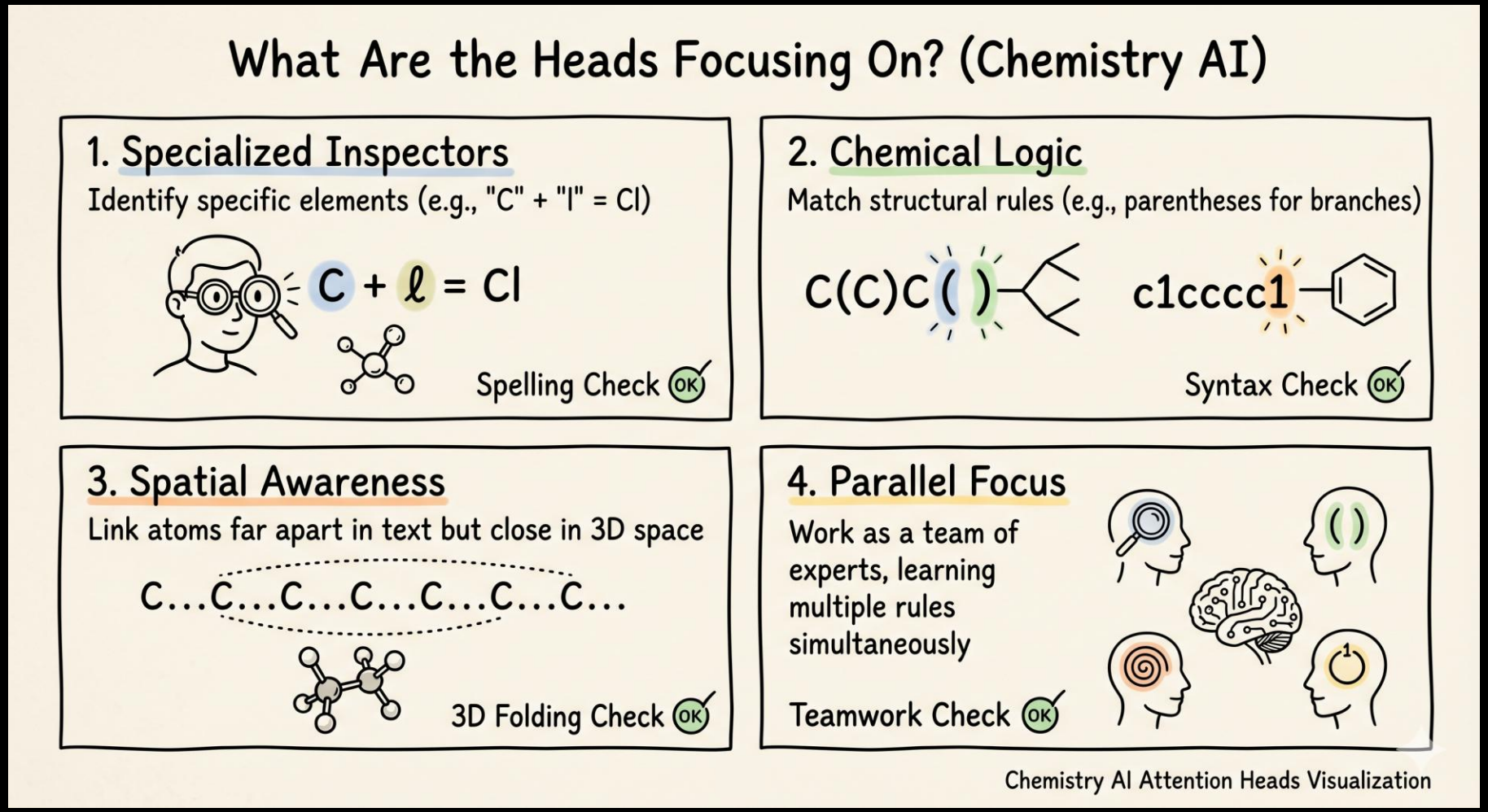
- Dataset Preprocessing
  - https://github.com/ycliu0214/MD-CADD-Courseware/blob/main/Jupyter-Notebook/Dataset_preprocessing.ipynb
- FreeMol-Gen
  - Training Demo: https://github.com/ycliu0214/MD-CADD-Courseware/blob/main/Jupyter-Notebook/FreeMol_Gen.ipynb
  - Inference Demo: https://github.com/ycliu0214/MD-CADD-Courseware/blob/main/Jupyter-Notebook/FreeMol_Gen_rst.ipynb
- ZINC-250k, Vocabulary Dictionary and Model Weights (packages.zip)
  - https://github.com/ycliu0214/MD-CADD-Courseware/blob/main/Jupyter-Notebook/packages.zip

Tokenizer & Embeddings

- Tokenizer (Character-Level)
  - Splits SMILES strings into individual characters (e.g., C, O, =).
  - Keeps the vocabulary dictionary extremely small and simple for beginners to grasp.
- Embeddings (Word Vectors)
  - Converts characters into high-dimensional "spatial coordinates.“
  - Gives lifeless symbols their initial mathematical meaning.
- Positional Encoding
  - Adds "seat numbers" to each token so the AI can distinguish sequence order (e.g., COC vs. OCC).
  - Transformers read all characters in parallel for speed.



Multi-Head Attention



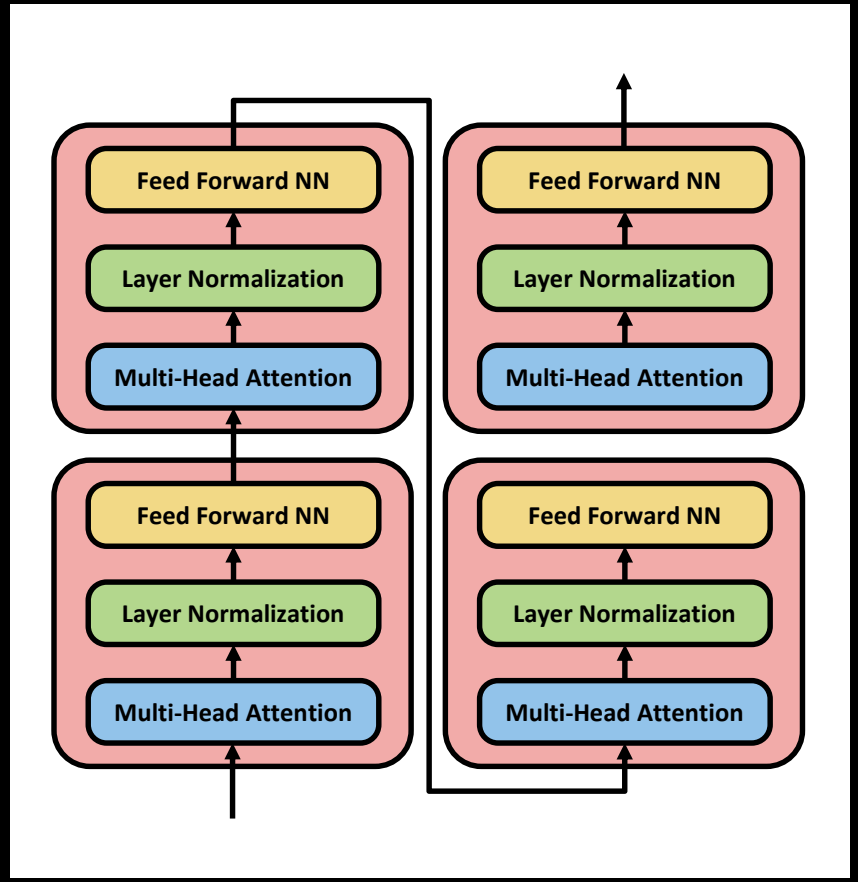
AI Generation [Google. (2026). *Gemini (Nano Banana 2)*.]

Normalization & Feed Forward Networks

- Layer Normalization (The Stabilizer)
  - Prevents extreme values during massive calculations.
  - Keeps the learning process stable and balanced.
- Feed Forward Network (The Reasoning Center)
  - Independent Processing: Analyzes each atom individually based on context.
  - Non-Linear Logic (ReLU): Digests complex chemical rules.
  - Feature Expansion: Temporarily expands dimensions (128 → 256) to refine hidden features.

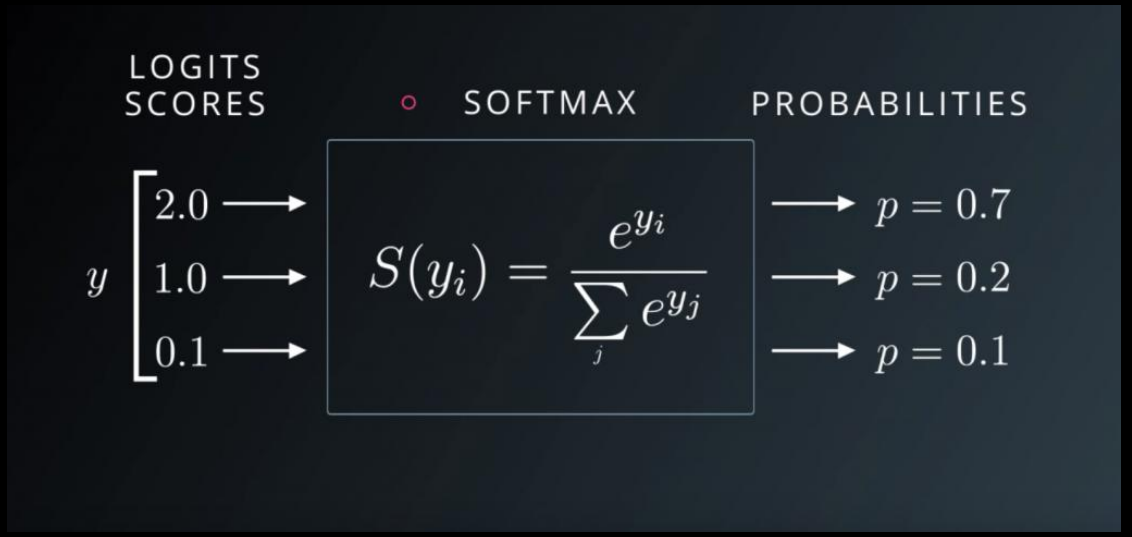
Stacking Transformer Blocks

- Cognitive Progression (From Letters to Molecules)
  - Early Block: Learn basic chemical spelling (e.g., C=O, Cl).
  - Middle Block: Recognize structural groups (e.g., carbon rings).
  - Deep Block: Understand global stability and physical properties.
- Abstract Feature Extraction (Refining Knowledge)
  - Iterative Refinement: Continuously upgrade values through each block.
  - Dimensional Transformation: Convert 1D text into a mathematical model rich in 3D and bonding context.



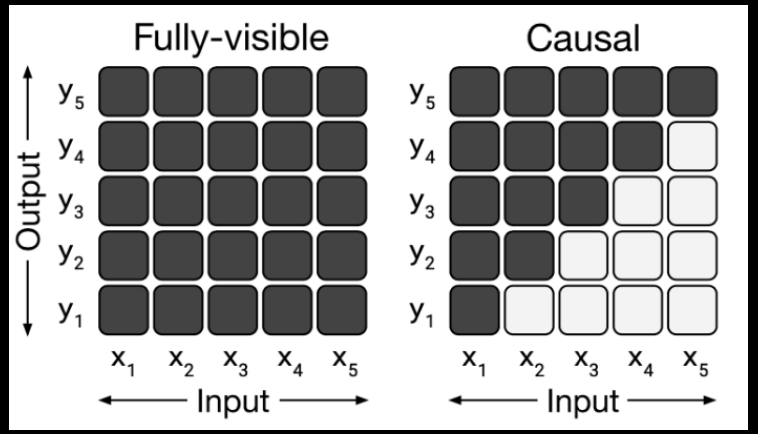
Softmax & Output Layer

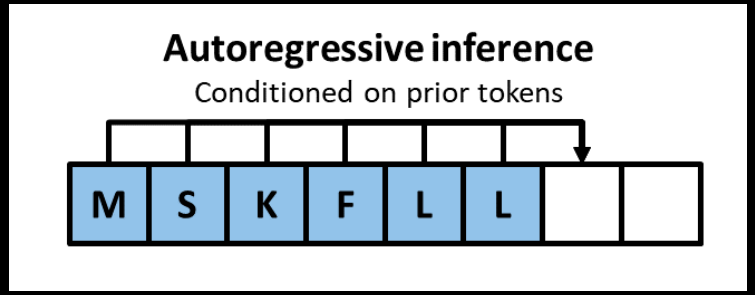
- The Output Layer (Translating Abstraction):
  Projects complex AI calculations back to the character dictionary size.
- Raw Scoring (Logits):
  Assigns a raw, unnormalized mathematical score to every character.
- Softmax Transformation (Standardization):
  Converts raw scores into exact percentages totaling 100%.
- Probability Distribution:
  Amplifies high scores to create a clear "probability pie chart."



Training & Inference

- Causal Masking | No Peeking
  - Hides future tokens during training.
  - Ensures predictions rely strictly on past context.
- Teacher Forcing | Guided Learning
  - Uses correct answers as inputs during training.
  - Prevents cascading errors and speeds up learning.
- Autoregressive Inference | Step-by-Step Generation
  - Generates the molecule one piece at a time.
  - Feeds each new piece back into the loop for the next prediction.
- Empirical Convergence | Mastering the Puzzle
  - Overcomes the massive complexity of chemical combinations.
  - Boosts valid molecule generation from near 0% (random) to >90%.

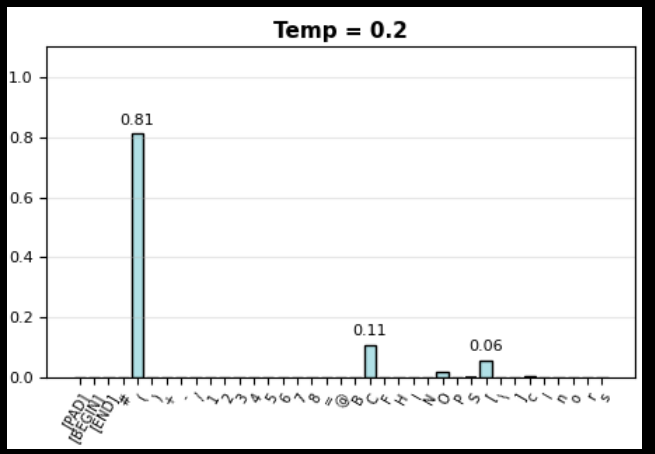


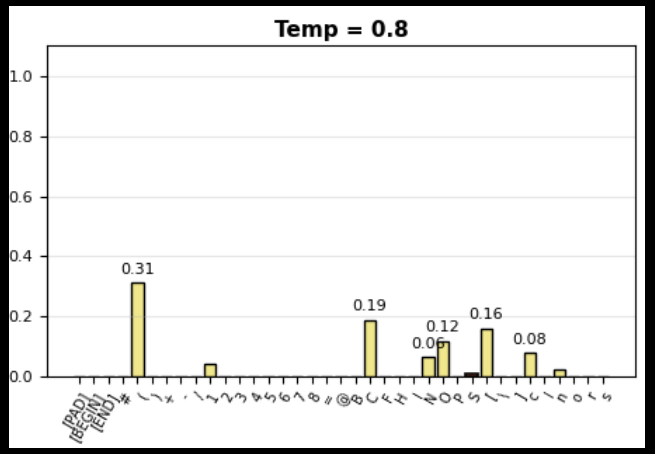


Bonus: Temperature & AI Hallucinations

- Temperature Sampling | The Creativity Dial
  - Adjusts the probability "pie chart" right before the final choice.
- Conservative vs. Creative | Balancing Act
  - Low Temp: Safe, common, and highly valid structures.
  - High Temp: Rare, novel, but riskier structures.
- AI Hallucinations | The Danger Zone
  - Too much "heat" causes impossible physics (e.g., carbon with 5 bonds).
- The Reality Check | AI vs. RDKit
  - AI: The creative brainstormer proposing millions of ideas.
  - RDKit: The strict referee filtering out invalid chemistry.

$$p_i = \frac{e^{\frac{z_i}{T}}}{\sum_j e^{\frac{z_j}{T}}}$$





Hyperparameters Summary

- Sequence & Vocabulary
  - vocab_size (37): The total number of unique characters the database (including SMILES symbols and [BEGIN], [END], [PAD]).
  - maxlen (128): The maximum length of SMILES that the model can process at one time is estimated to be 128 characters for this dataset after preserving buffer length.
- Architecture Scale
  - num_layers (4): The number of stacked Transformer blocks, defining the "depth of chemical cognition.“
  - embed_dim (128): The size of the initial mathematical spatial coordinates for each character.
  - ff_dim (256): The expanded dimensional space within the Feed Forward Network for deeper feature reasoning.
  - num_heads (4): The number of parallel "specialized inspectors" in the Attention mechanism.
- Training & Inference
  - batch_size & epochs: Determines how many molecules the AI studies at once, and how many times it reviews the entire dataset.
  - temperature (0.8~1.2, suggest 1.0): The dial used during inference to balance between conservative validity and creative drug design.

Last Question

Can AI generate new molecules with specific scaffolds and properties?

JCIM JOURNAL OF CHEMICAL INFORMATION AND MODELING

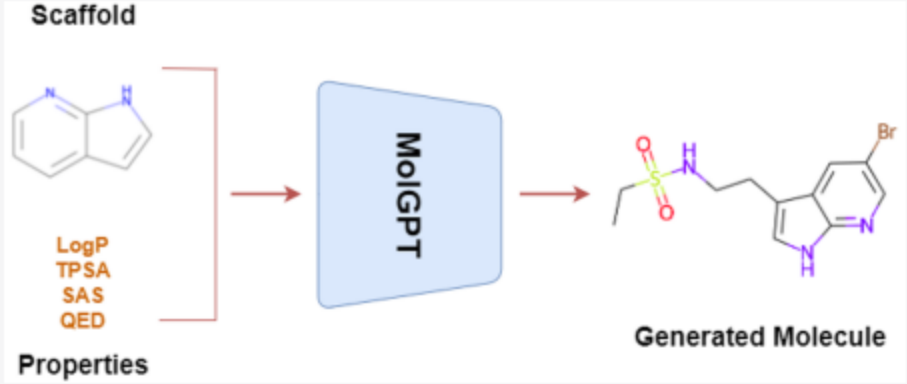
pubs.acs.org/jcim Article

MolGPT: Molecular Generation Using a Transformer-Decoder Model

Viraj Bagal, Rishal Aggarwal, P. K. Vinod, and U. Deva Priyakumar*

Cite This: *J. Chem. Inf. Model.* 2022, 62, 2064–2076 Read Online

ACCESS | Metrics & More | Article Recommendations | Supporting Information



ABSTRACT: Application of deep learning techniques for *de novo* generation of molecules, termed as inverse molecular design, has been gaining enormous traction in drug design. The representation of molecules in SMILES notation as a string of characters enables the usage of state of the art models in natural language processing, such as Transformers, for molecular design in general. Inspired by generative pre-training (GPT) models that have been shown to be successful in generating meaningful text, we train a transformer-decoder on the next token prediction task using masked self-attention for the generation of druglike molecules in this study. We show that our model, MolGPT, performs on par with other previously proposed modern machine learning frameworks for molecular generation in terms of generating valid, unique, and novel molecules. Furthermore, we demonstrate that the model can be trained conditionally to control multiple properties of the generated molecules. We also show that the model can be used to generate molecules with desired scaffolds as well as desired molecular properties by conditioning the generation on scaffold SMILES strings of desired scaffolds and property values. Using saliency maps, we highlight the interpretability of the generative process of the model.